



PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 3

Introduction to OOPS Programming

SESSION / ASSIGNMENT – 1

TASKS :

1. Write a simple C program tasklist_c.c that allows you to add up to 5 tasks (as strings) to a global array and print all tasks using a for loop.



```
#include <stdio.h>
#include <string.h>

char tasks[5][100];
int taskCount = 0;

int main()
{
    int i;

    printf("Enter up to 5 tasks:\n");

    for(i = 0; i < 5; i++)
    {
        printf("Enter Task %d: ", i + 1);
        fgets(tasks[i], 100, stdin);

        // Remove the newline added by fgets()
        tasks[i][strcspn(tasks[i], "\n")] = '\0';

        taskCount++;
    }

    printf("\nTask List:\n");

    for(i = 0; i < taskCount; i++)
    {
        printf("%d. %s\n", i + 1, tasks[i]);
    }
}
```

```
    return 0;
}
```

OUTPUT:

```
Enter up to 5 tasks:
Enter Task 1: bug report
Enter Task 2: presentation
Enter Task 3: file handling
Enter Task 4: assignments
Enter Task 5: teaching

Task List:
1. bug report
2. presentation
3. file handling
4. assignments
5. teaching
```

2. Modify your tasklist_c.c to add a function markTaskDone(int index) that sets the selected task to 'DONE' in the array, then print the updated list.

Hint: Use a separate status array or append ' - DONE' to the task string.



```
#include <stdio.h>
#include <string.h>

char tasks[5][100];
int taskStatus[5] = {0};
int taskCount = 0;

void markTaskDone(int index)
{
    taskStatus[index] = 1;
}

int main()
{
    int i;
    int taskNumber;

    printf("Enter 5 tasks:\n");

    for(i = 0; i < 5; i++)
    {
        printf("Enter Task %d: ", i + 1);
        fgets(tasks[i], 100, stdin);

        tasks[i][strcspn(tasks[i], "\n")] = '\0';
    }
}
```

```

        taskCount++;
    }

    printf("\nEnter the task number to mark as done: ");
    scanf("%d", &taskNumber);

    markTaskDone(taskNumber - 1);

    printf("\nUpdated Task List:\n");

    for(i = 0; i < taskCount; i++)
    {
        if(taskStatus[i] == 1)
        {
            printf("%d. %s - DONE\n", i + 1, tasks[i]);
        }
        else
        {
            printf("%d. %s\n", i + 1, tasks[i]);
        }
    }

    return 0;
}

```

OUTPUT:

```

Enter 5 tasks:
Enter Task 1: Training
Enter Task 2: Bug Report
Enter Task 3: Presentation
Enter Task 4: Side Work
Enter Task 5: Planning Schedule

Enter the task number to mark as done: 1

Updated Task List:
1. Training - DONE
2. Bug Report
3. Presentation
4. Side Work
5. Planning Schedule

```

3. Create a Task class in your favorite OOP language (Java, Python, or C++) with properties: title and isDone. Add a method markDone() that sets isDone to true, and a method display() that prints the task with its status.



```
#include <iostream>
#include <string>
using namespace std;

class Task
{
public:
    string title;
    bool isDone;

    Task(string taskTitle)
    {
        title = taskTitle;
        isDone = false;
    }

    void markDone()
    {
        isDone = true;
    }

    void display()
    {
        if(isDone)
        {
            cout << title << " - DONE" << endl;
        }
        else
        {
            cout << title << " - NOT DONE" << endl;
        }
    }
};

int main()
{
    Task task("Complete C++ Assignment");

    task.display();

    task.markDone();

    task.display();
}
```

```
    return 0;
}
```

OUTPUT:

```
Complete C++ Assignment - NOT DONE
Complete C++ Assignment - DONE
```

4. Build a simple TaskList class/object that stores multiple Task objects and provides addTask(title), markTaskDone(index), and showTasks() methods. Demonstrate adding 3 tasks, marking one as done, and displaying all tasks with their statuses.



```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

class Task
{
public:
    string title;
    bool isDone;

    Task(string taskTitle)
    {
        title = taskTitle;
        isDone = false;
    }

    void markDone()
    {
        isDone = true;
    }

    void display()
    {
        if(isDone)
        {
            cout << title << " - DONE" << endl;
        }
        else
        {
            cout << title << " - NOT DONE" << endl;
        }
    }
};
```

```

class TaskList
{
private:
    vector<Task> tasks;

public:
    void addTask(string title)
    {
        tasks.push_back(Task(title));
    }

    void markTaskDone(int index)
    {
        tasks[index].markDone();
    }

    void showTasks()
    {
        for(int i = 0; i < tasks.size(); i++)
        {
            cout << i + 1 << ". ";
            tasks[i].display();
        }
    }
};

int main()
{
    TaskList myTasks;

    myTasks.addTask("Complete assignment");
    myTasks.addTask("Study C++");
    myTasks.addTask("Practice coding");

    myTasks.markTaskDone(1);

    myTasks.showTasks();

    return 0;
}

```

OUTPUT:

```

1. Complete assignment - NOT DONE
2. Study C++ - DONE
3. Practice coding - NOT DONE

```

5. Compare your procedural C version and OOP TaskList version: List 3 problems you faced in the C version that were solved by using classes and methods in the OOP version.



C	C++
Tasks and their status had to be stored in separate arrays.	A Task object stores the title and status together.
Functions and data were separate, making the program harder to organize as it grew.	Classes combine data and methods such as <code>markDone()</code> and <code>display()</code> .
Managing multiple tasks required manually handling arrays and indexes.	TaskList manages multiple Task objects and provides methods like <code>addTask()</code> and <code>markTaskDone()</code> .